สรุปผลการประเมินระบบ

Agentic AI ด้านการเงินส่วนบุคคลสำหรับผู้ใช้ชาวไทย

Pakkaphon (รหัสนักศึกษา 66070146) คณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

สรุปผลใน 1 หน้า

ระบบเป็นสถาปัตยกรรมแบบหลาย Agent (Multi-Agent Architecture) ที่มี Router Agent จำแนกเจตนาของคำถาม แล้วส่งต่อให้ Agent เฉพาะทาง 6 ตัว หรือ General Chat ระบบที่ใช้งานจริงใช้ Google Gemini 3.5 Flash คณะผู้จัดทำประเมินระบบ 4 ด้าน ผลโดยสรุปเป็นดังตาราง

การทดสอบ	คำถามที่ต้องการตอบ	ผลลัพธ์
สถาปัตยกรรม	แบบใดทำงานสำเร็จมากที่สุด	บน DeepSeek แบบ Single Agent สำเร็จ 98.9% ระบบจริง (แบบ Hub-and-Spoke ที่ Multi-intent Routing) สำเร็จ 92.2% (แบบไม่แยกงาน 87.2%) โดยใช้ token น้อยกว่าแบบ Single Agent 3.3 เท่า
Multi-intent Routing	รองรับคำถามที่ต้องใช้หลาย Agent ได้หรือไม่	Router ที่ใช้งานจริงจำแนกถูก 205 จาก 215 ข้อ (95.3%) ทั้งคำถามเรื่องเดียวและหลายเรื่อง คำถามที่ต้องใช้หลาย Agent สำเร็จประมาณ 86.1% เทียบกับ 61.1% ของแบบไม่แยกงาน ($p = 0.012$)
ระดับการคิดของ Router	ใช้ระดับต่ำสุดแล้วยังแม่นยำหรือไม่	แม่นยำไม่ต่างกัน (95.9% เทียบกับ 96.2%) และเร็วขึ้นเกือบเท่าตัว (1.36 เทียบกับ 2.59 วินาที)
การค้นคืนเอกสาร	โมเดลเวกเตอร์แทนความหมายตัวใดดีที่สุด	Gemini ค้นคืนได้ดีที่สุด (nDCG@10 0.805) แต่คำตอบสุดท้ายของ 3 โมเดลที่ดีที่สุดถูกต้องใกล้เคียงกัน (92.5–94.2%)
การคำนวณภาษี	ภาษีถูกต้องตามกฎปี 2569 หรือไม่	ถูกต้อง 26 จาก 26 กรณี ถ้าไม่ใช้เครื่องมือคำนวณ โมเดลตอบถูก 7 จาก 8 กรณี และยังใช้เพดานประกันสังคมแบบเก่า

ข้อสรุปหลัก แบบ Hub-and-Spoke ที่มี Multi-intent Routing ปิดช่องว่างกับแบบ Single Agent ได้เกือบหมด โดยยังประหยัด token สำหรับคำถามเรื่องเดียว ส่วนการตั้งค่า Router การเลือกโมเดลเวกเตอร์ และการคำนวณภาษี ผลการทดลองรองรับการออกแบบที่ใช้อยู่ทั้งหมด

ตัวเลขทุกตัวในรายงานนี้มาจากไฟล์ผลการทดลองโดยตรง และทุกรอบที่ล้มเหลวในการทดสอบสถาปัตยกรรมได้อ่านตรวจด้วยมือก่อนนับผล ข้อมูลรายข้อทั้งหมดอยู่ในรายงานฉบับเต็มภาษาอังกฤษ (evaluation-report.pdf)

1 วิธีทดสอบโดยย่อ

ระบบสร้างโมเดลภาษาผ่านส่วนสร้างโมเดลกลางที่รองรับ Google Gemini, Ollama และ OpenRouter จึงเปลี่ยนโมเดลได้ด้วยการตั้งค่าเพียงจุดเดียว การทดสอบสถาปัตยกรรมใช้โมเดล 2 ตระกูล คือ gpt-oss:20b และ DeepSeek V4.1 Flash ส่วนการทดสอบอื่นใช้ Gemini ตัวเดียวกับระบบจริง

ชุดข้อมูล	จำนวน	รายละเอียด
งานทดสอบสถาปัตยกรรม	60	คำถามเรื่องเดียว 40 งาน ใช้หลาย Agent 12 งาน และคำถามกว้างที่ต้องถามกลับ 8 งาน
คำถามจำแนกเจตนา	185	เขียนใหม่ 150 ข้อ และชุดเดือนกันยายน 35 ข้อ
ข้อความที่ถามหลายเรื่อง	30	หลายเรื่อง 22 ข้อ (2–5 เรื่อง) และคำถามเดียวที่ฟังคล้ายหลายเรื่อง 8 ข้อ
คำถามค้นคืนเอกสาร	120	ใช้ภาษาแบบผู้ใช้ พร้อมไฟล์และคำสำคัญที่ใช้ตัดสินว่าเอกสารเกี่ยวข้อง
กรณีภาษี	26	กรณีทั่วไป 20 กรณี และกรณีตามกฎปี 2569 อีก 6 กรณี

การเปรียบเทียบใช้ช่วงความเชื่อมั่น 95% แบบ Wilson และใช้การทดสอบ McNemar แบบแม่นตรงเมื่อเทียบงานเดียวกันในรอบเดียวกัน ค่า pass^3 คือสัดส่วนงานที่สำเร็จครบทั้ง 3 รอบ ซึ่งใช้วัดความสม่ำเสมอ เวลาที่รายงานทั้งหมดเป็นเวลาตอบโดยทั่วไป (มัธยฐาน) คือค่าที่อยู่ตรงกลางเมื่อเรียงเวลาของทุกรอบจากน้อยไปมาก ซึ่งไม่ถูกดึงขึ้นจากรอบที่ช้าผิดปกติเหมือนค่าเฉลี่ย

2 การเปรียบเทียบสถาปัตยกรรม

คณะผู้จัดทำสร้างสถาปัตยกรรม 4 แบบจาก Agent และเครื่องมือชุดเดียวกัน ได้แก่ แบบ Single Agent ที่มีเครื่องมือทั้ง 27 รายการ แบบ Hub-and-Spoke ที่ใช้ในระบบจริง แบบ Hierarchical และแบบ Peer-to-Peer ทุกแบบทำงาน 60 งาน งานละ 3 รอบ งานหนึ่งนับว่าสำเร็จเมื่อส่งถูก Agent เรียกเครื่องมือที่จำเป็นครบ ตอบตัวเลขถูก และบันทึกข้อมูลถูกต้อง

สถาปัตยกรรม (DeepSeek)	สำเร็จรวม	ใช้หลาย Agent	pass^3	Token ขาเข้า	เวลาตอบทั่วไป (วินาที)
Single Agent	98.9%	100.0%	96.7%	42,555	3.6
Hub-and-Spoke + Multi-intent Routing (ระบบจริง)*	92.2%	86.1%	90.0%	12,946	4.4
แบบ Peer-to-Peer	91.7%	66.7%	88.3%	14,807	5.6
Hub-and-Spoke (ไม่แยกงาน)	87.2%	61.1%	83.3%	11,649	4.4
แบบ Hierarchical	85.0%	55.6%	80.0%	12,280	5.3

* แถวระบบจริงวัด Multi-intent Routing จาก 6 งานที่ต้องใช้หลาย Agent ซึ่งแบบไม่แยกงานล้มเหลวอย่างน้อย 1 รอบ (18 รอบ) ส่วนงานอื่นผ่านเส้นทางเดียวกันจึงใช้ผลร่วมกัน (ดูหัวข้อ Multi-intent Routing) ระบบจริงยังต่ำกว่าแบบ Single Agent อย่างมีนัยสำคัญ (14 คู่เทียบกับ 2 คู่, $p = 0.004$) แต่ pass^3 อยู่ที่ 90.0% เทียบกับ 83.3% ของแบบไม่แยกงาน และเวลาเท่ากัน

บน gpt-oss:20b (ไม่ได้ทดสอบ Multi-intent Routing) ผลต่างออกไป แบบ Single Agent และแบบ Hub-and-Spoke เสมอกัน (83.9% และ 83.3%, $p = 1.0$) และแบบ Hub-and-Spoke สม่ำเสมอที่สุด (pass^3 78.3% เทียบกับ 73.3%)

ข้อสรุป กับโมเดลที่เก่ง แบบ Single Agent ชนะเพราะใช้เครื่องมือข้ามด้านได้ในคำถามเดียว ส่วนแบบ Hub-and-Spoke ที่ไม่แยกงานส่งคำถามให้ Agent ตัวเดียว จึงตอบคำถามที่ต้องใช้หลาย Agentไม่ครบ ข้อดีของแบบ Hub-and-Spoke คือใช้ token น้อยกว่า 3.6 เท่า และไม่เสียเปรียบเมื่อใช้โมเดลที่อ่อนกว่า

3 Multi-intent Routing

Multi-intent Routing ให้ Router Agent แยกข้อความที่ถามหลายเรื่องเป็นงานย่อย งานละ 1 Agent ตามลำดับที่ผู้ใช้ถาม โดยไม่จำกัดจำนวนงาน Agent ทำงานทีละตัว ตัวหลังเห็นคำตอบของตัวก่อนหน้า แล้วระบบรวมคำตอบเป็นข้อความเดียว คำถามเรื่องเดียวส่งให้ Agent ตัวเดียวตามปกติ

กลไกนี้อยู่ใน Router Agent ทั้งหมด คณะผู้จัดทำจึงทดสอบดังนี้

การทดสอบ	ผลลัพธ์
Router ที่ใช้งานจริง 215 ข้อ	ถูก 205/215 = 95.3% (CI 91.7–97.5%): คำถามเรื่องเดียว 175/185 (94.6%, ไม่ต่างจาก 95.7% ของคำสั่งที่ไม่มี Multi-intent Routing, $p = 0.69$, แยกเกิน 1 ข้อ) หลายเรื่อง 22/22 ถูกทุก Agent และลำดับ และคำถามเดียวที่ฟังคล้ายหลายเรื่อง 8/8
ทำงานจริงบน DeepSeek	ทดสอบ 6 งานที่ต้องใช้หลาย Agent ซึ่งแบบไม่แยกงานล้มเหลวอย่างน้อย 1 รอบ (18 รอบ) ส่วนงานอื่นใช้ผลร่วมกัน

สถาปัตยกรรม (DeepSeek)	สำเร็จรวม	ใช้หลาย Agent	ใช้หลาย Agent pass^3
Hub-and-Spoke (ไม่แยกงาน)	87.2%	61.1%	6/12
แบบ Hub-and-Spoke + Multi-intent Routing (ประมาณการ)	92.2%	86.1%	10/12
Single Agent	98.9%	100.0%	12/12

ใน 18 คู่นี้ Multi-intent Routing สำเร็จในจุดที่แบบไม่แยกงานล้มเหลว 10 ครั้ง และกลับกันเพียง 1 ครั้ง ($p = 0.012$) ข้อแลกเปลี่ยนคือข้อความที่ถามหลายเรื่องใช้ทรัพยากรมากขึ้น โดยเรียกโมเดลเพิ่มจาก 3.4 เป็น 5.8 ครั้ง token จาก 11.2k เป็น 24.1k และเวลาจาก 9.0 เป็น 13.0 วินาที ส่วนคำถามเรื่องเดียวไม่ได้รับผลกระทบ

ข้อควรระวัง ตัวเลข 86.1% เป็นค่าประมาณ เพราะไม่ได้ทดสอบ 6 งานที่ต้องใช้หลาย Agent ซึ่งแบบไม่แยกงานสำเร็จทุกรอบ ความล้มเหลวที่เหลือ 5 รอบอ่านตรวจด้วยมือแล้ว คือส่งคำถามเรื่องการออมไปผิด Agent 2 รอบ และ Agent ภาษีของ DeepSeek ตอบว่าง 3 รอบ

4 ระดับการคิดของ Router Agent

บทที่ 3 ระบุว่า Router Agent ใช้ระดับการคิดต่ำสุดของ Gemini โดยไม่เสียความแม่นยำ การทดสอบนี้ใช้คำถาม 185 ข้อ ข้อละ 2 รอบ ผลการทดสอบเบื้องต้นพบว่าระดับ low และ minimal ใช้ token สำหรับการคิดเป็นศูนย์เท่ากัน จึงเทียบ minimal กับค่าเริ่มต้นของโมเดล

โมเดล	ระดับการคิด	ความแม่นยำ	เวลาตอบทั่วไป (วินาที)	Token การคิด
gemini-3.5-flash	minimal (ระบบจริง)	95.9%	1.36	0
gemini-3.5-flash	ไม่ตั้งค่า	96.2%	2.59	272
gemini-3.5-flash-lite	minimal	93.8%	0.88	0
gemini-3.5-flash-lite	high	96.5%	1.93	469

ข้อสรุป ระดับ minimal ผ่านเกณฑ์ไม่ด้อยกว่า ความแม่นยำต่างกัน -0.3 จุด ช่วงความเชื่อมั่นอยู่ที่ -2.2 ถึง $+1.6$ จุด ซึ่งอยู่เหนือเกณฑ์ -3 จุดทั้งช่วง และเร็วขึ้นประมาณ 1.26 วินาที รองรับข้อความในบทที่ 3 ส่วน Flash-Lite แม่นยำน้อยกว่า 2.2 จุด จึงไม่นำมาใช้ ส่วนคำสั่งของ Router ที่ใช้งานจริงซึ่งมี Multi-intent Routing จำแนกคำถามเรื่องเดียวถูก 94.6% ไม่ต่างจากผลในตารางนี้อย่างมีนัยสำคัญ

5 การค้นคืนเอกสาร

คณะผู้จัดทำเทียบวิธีค้นคืน 5 แบบ ด้วยคำถาม 120 ข้อ บนฐานความรู้ที่ตัดข้อความแบบเดียวกับระบบจริง จากนั้นให้ Gemini ตอบคำถามจากเอกสาร 3 ชิ้นแรกของแต่ละวิธี และให้ DeepSeek ซึ่งเป็นโมเดลต่างตระกูลเป็นผู้ตัดสิน ผู้ตัดสินให้ผลตรงกับการอ่านตรวจด้วยมือ 28 จาก 30 คำตอบ

วิธีค้นคืน	nDCG@10	Recall@3	คำตอบถูกต้อง	ตรงกับแหล่งอ้างอิง
gemini-embedding-001 (ระบบจริง)	0.805	0.900	93.3%	88.3%
BAAI/bge-m3	0.724	0.842	94.2%	82.5%
multilingual-e5-large-instruct	0.722	0.825	92.5%	86.7%
BM25	0.568	0.683	—	—
MiniLM	0.394	0.433	—	—

ข้อสรุป Gemini ค้นคืนได้ดีกว่าทุกวิธีอย่างมีนัยสำคัญ จึงใช้ต่อในระบบจริง ส่วน MiniLM ไม่เหมาะกับภาษาไทย ระบบจึงใช้ bge-m3 เป็นตัวสำรองบนเครื่อง อย่างไรก็ตาม ความถูกต้องของคำตอบสุดท้ายของ 3 วิธีแรกไม่ต่างกันอย่างมีนัยสำคัญ เพราะโมเดลมักตอบถูกได้แม้เอกสาร 3 ชิ้นจะเกี่ยวข้องเพียงบางส่วน

6 การคำนวณภาษี

ตัวคำนวณภาษีและคำสั่งของ Tax Agent ใช้กฎปี 2569 ชุดทดสอบจึงมี 6 กรณีที่ถ้าใช้กฎของปีก่อนจะได้คำตอบผิด

การทดสอบ	ผล	หมายเหตุ
ความถูกต้องผ่าน Tax Agent	26/26	ไม่มีข้อผิดพลาด ช่วงความเชื่อมั่นของอัตราผิดพลาดอยู่ที่ 0–12.9%
ใช้เครื่องมือคำนวณ	8/8	ถูกต้องทุกกรณี
ไม่ใช้เครื่องมือ (อ่านด้วยมือ)	7/8	ผิดกรณีเพดานรวม RMF กับ SSF (ตอบ 467,000 แทน 407,000 บาท) และยังใช้เพดานประกันสังคม 9,000 บาทแบบเก่า
การตรวจข้อมูลผิด 10 กรณี	10/10	ระบบตรวจอัตโนมัติให้ 9/10 แต่เมื่ออ่านแล้วกรณีที่ไม่ผ่านตอบถูกต้อง

ข้อสรุป ยังควรใช้เครื่องมือคำนวณภาษี ไม่ใช่เพราะโมเดลคิดเลขไม่ได้ แต่เพราะเครื่องมือทำให้ทุกคำตอบถูกต้องและเป็นกฎปัจจุบันเสมอ

7 ข้อจำกัด

- ไม่ได้ทดสอบสถาปัตยกรรมซ้ำบน Gemini 3.5 Flash ด้วยข้อจำกัดด้านงบประมาณ คณะผู้จัดทำเลือก Gemini สำหรับระบบจริงเพราะเสถียรที่สุดในการทดสอบที่ใช้ Gemini คือ ภาษีถูก 26 จาก 26 กรณี จำแนกคำถาม 185 ข้อโดยไม่มีครั้งใดล้มเหลว และไม่พบคำตอบว่าง ขณะที่ DeepSeek ตอบว่างเป็นระยะ และ gpt-oss:20b เคยถามกลับเป็นภาษาจีน
- ผลของ Multi-intent Routing ในคำถามที่ต้องใช้หลาย Agentเป็นค่าประมาณ
- ชุดข้อมูลทั้งหมดเขียนขึ้นเอง ไม่ได้มาจากข้อความของผู้ใช้จริง
- ยังไม่มีการประเมินโดยผู้เชี่ยวชาญ การทดสอบกับผู้ใช้จริง หรือการทดสอบเชิงโจมตี
- เวลาตอบวัดจากเครื่องของคณะผู้จัดทำ ไม่ได้วัดจากเซิร์ฟเวอร์จริง